

## Investigating an Unknown *C. elegans* Gene with a Human Ortholog

### Which Worm Genes Have Human Analogs?

A **model organism** is a species that biologists study to gain insights into human biology. In particular, model organisms are widely used to study analogs of human diseases when experiments on humans are infeasible. For example, model organisms include the bacterium *Escherichia coli* (to study the human microbiome), the simple eukaryote *Dictyostelium discoideum* (to study cell communication and programmed cell death), guinea pigs (to study bacterial infections), and mice (to study obesity and many other diseases). Although studying model organisms is very useful, extrapolating results from flies and mice to humans should be done with caution.

In this Bioinformatics Application Challenge, we will analyze the non-parasitic nematode *Caenorhabditis elegans* (figure below), which biologists study to understand the genetic control of development. It was proposed as a model for neuronal development by Sydney Brenner in 1963 and was the first multicellular organism to have its genome sequenced. Furthermore, it is now the only organism whose *connectome* (neural wiring diagram) is known.



**Figure:** *C. elegans*.

*C. elegans* has many cellular and molecular commonalities with humans. For example, according to **OrthoList**, a compendium of *C. elegans* genes, there are over 7,500 protein-coding genes in *C. elegans* that have **orthologs**, or evolutionarily related genes, in humans. Furthermore, since it is small (adults are only 1 mm long), large numbers of *C. elegans* organisms can be stored in a lab. Its life cycle (the time from birth to generation of offspring) is only about three days and its life span is only about three weeks, which shortens the amount of time needed for each experiment. *C. elegans* is also transparent, which simplifies analysis of cell division and cell lineage. Finally, *C. elegans* is non-parasitic and feeds on bacteria, making it safe and cheap to maintain in a laboratory.

Although there are already over 7,500 known orthologs between *C. elegans* and humans, biologists are constantly searching for elusive similarities between other *C. elegans* and human genes. If we are able to find human proteins that are similar in function to worm sequences, then we can study the worm counterpart in a laboratory and extrapolate the experimental results to humans (with some further experiments, of course).

In this Bioinformatics Application Challenge, you will analyze a short *C. elegans* gene whose function remained mysterious for a long time. Your goal is to determine the function of this gene and see whether it has anything to do with human proteins, which represent some of the most valuable drug target in humans. But before we plunge into analyzing worm genes, let's recall some basic facts about HMMs.

### **Background on Hidden Markov Models (HMMs).**

An HMM is an abstract probabilistic machine that emits symbols. It is specified by the following parameters:

1. An alphabet of emitted symbols,  $\Sigma$
2. A set *States* of hidden states.
3. A  $|States| \times |States|$  matrix *Transition* of transition probabilities, which contains the probabilities of transitioning from each hidden state to every other hidden state.
4. A  $|States| \times |\Sigma|$  matrix *Emission*, which contains the probabilities of every hidden state emitting a certain symbol from the alphabet.

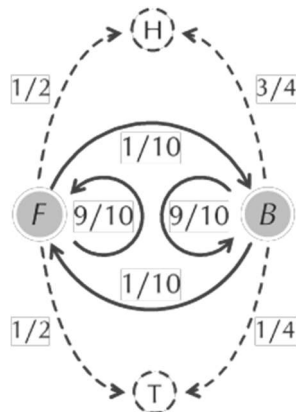
An HMM is also characterized by its **starting probabilities**, where the starting probability of a hidden state is the probability that the HMM starts emitting symbols from this state in the initial moment. Whereas in the course we assumed that the starting probabilities are the same for all hidden states, in this application challenge the starting probabilities may vary between states.

We will start by analyzing the crooked casino using HMMs and then consider applications of HMMs in biology.

## Analyzing the Crooked Casino

Given a string emitted by an HMM, the Viterbi algorithm solves the Decoding Problem, i.e., it finds the most likely sequence of hidden states that generated this string.

To solve the Decoding Problem for a given HMM, we will use Viterbi.py, a Python script that we provide for this purpose. Viterbi.py simulates the HMM described by the following *HMM diagram* that models the crooked casino (with states *F* and *B* emitting symbols H and T).



In the crooked casino, you are placing bets on whether a dealer's coin will land on heads or tails. However, the dealer does not always use a fair coin with the probability of landing on heads equal to 0.5. Sometimes the dealer will switch out the coin and use a biased coin (with the probability of landing on head equal to 0.75) without you noticing. Here is the description of the HMM modelling the crooked dealer.

1.  $\Sigma$ : Heads (H) and Tails (T)
2. *States*: Fair (F) and Biased (B)
3. *Transition* matrix:

|                              | Transition To: <b>Fair</b> | Transition To: <b>Biased</b> |
|------------------------------|----------------------------|------------------------------|
| Current state: <b>Fair</b>   | 0.90                       | 0.10                         |
| Current state: <b>Biased</b> | 0.10                       | 0.90                         |

4. *Emission* matrix

|                            | Emit: <b>Heads</b> | Emit: <b>Tails</b> |
|----------------------------|--------------------|--------------------|
| Current State: <b>Fair</b> | 0.50               | 0.50               |

Current State: **Biased** 0.75                      0.25

The starting probabilities are the same for all states.

The solution of the Decoding Problem is the most likely sequence of hidden states  $\pi$  that generated the emitted sequence  $x$  and the probability  $\Pr(x, \pi)$  that the HMM emits  $x$  and follows the sequence of hidden states  $\pi$ . In addition to outputting this data, Viterbi.py also outputs the scores of nodes in the **Viterbi graph**.

To get started with Viterbi.py, first click [here](#) to download it. Save this file to a directory on your computer where you will work with it. This file is written in Python, so in order to work with it, we will need to download Python from the [Python website](#) (choose version 3). To make running Python as smooth as possible, we suggest downloading [PyCharm Community Edition](#). Then open PyCharm, click "File" --> "Open" and select "Viterbi.py" from the directory where you saved it.

**Note:** You will not need to understand how to program in Python in order to complete this application challenge.

Run Viterbi.py (click the green triangle button in PyCharm). This will run Viterbi.py for the emitted string "THHHH" because of the line

**observations = ('tails', 'heads', 'heads', 'heads', 'heads')**

You can verify that the output is given by the following, which is divided into a table and a line following the table.

```
      0      1      2      3      4
Biased: 0.12500 0.08437 0.05695 0.03844 0.02594
Fair:    0.25000 0.11250 0.05062 0.02278 0.01025

(0.025949267578125004, ['Biased', 'Biased', 'Biased', 'Biased', 'Biased'])
```

### What does the table signify?

The table signifies the probability of each coin being used for each flip (using 0-based indexing) based on the emitted string ("THHHH"). The values in each cell are computed by multiplying the maximum of the previous column by the transition probability of the row and by the emission probability of the column (the first column uses the probabilities of starting in the corresponding state). The probability of most likely sequence of hidden states is the maximum of the final column, and is reproduced in the line below the table.

### Interpret the two items in the final line of output.

The number is the probability of the most likely hidden state sequence given the observed string of flips “THHHH”, and the list is the most likely hidden state sequence itself (in this case, only use of the Biased coin).

We now will change the second emitted symbol so that the emitted string is now "TTHHH" instead of "THHHH". To do so, change the line  
observations = ('tails', 'heads', 'heads', 'heads', 'heads') to the line  
observations = ('tails', 'tails', 'heads', 'heads', 'heads')

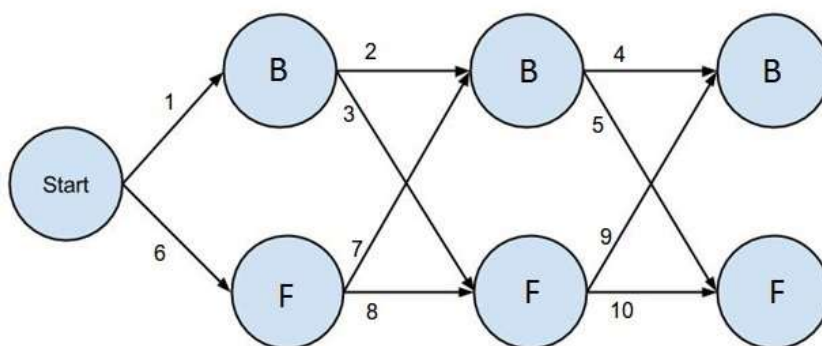
Then run Viterbi.py again.

**How did this change in the emitted symbols affect the probability of the most likely sequence of hidden states? How did it change the most likely sequence of hidden states? Interpret this result.**

The most likely sequence is now all Fair coin, with overall probability 0.0103 (which is smaller than the previous sequence).

After 5 coin flips, if we observed 3 heads we believe (and it is more likely) that the coin is fair (we don't have strong evidence that the coin is biased). If we had observed 4 heads, we would believe that the biased coin was used.

The figure below is the Viterbi graph for the crooked casino HMM emitting **HTH**.



**Compute the weights of enumerated edges in this graph (10 edges total).**

- 1 [Start --> B1]:  $P(\text{starting with Biased}) * P(\text{emit H} | \text{Biased}) = 0.5 * 0.75 = 0.375$
- 2 [B1 --> B2]:  $P(\text{transition Biased} \rightarrow \text{Biased}) * P(\text{emit T} | \text{B}) = 0.90 * 0.25 = 0.225$
- 3 [B1 --> F2]:  $P(\text{B} \rightarrow \text{F}) * P(\text{emit T} | \text{F}) = 0.1 * 0.5 = 0.05$
- 4 [B2 --> B3]:  $P(\text{B} \rightarrow \text{B}) * P(\text{emit H} | \text{B}) = 0.9 * 0.75 = 0.675$
- 5 [B2 --> F3]:  $P(\text{B} \rightarrow \text{F}) * P(\text{emit H} | \text{F}) = 0.1 * 0.5 = 0.05$
- 6 [Start --> F1]:  $P(\text{start F}) * P(\text{emit H} | \text{F}) = 0.5 * 0.5 = 0.25$
- 7 [F1 --> B2]:  $P(\text{F} \rightarrow \text{B}) * P(\text{emit T} | \text{B}) = 0.1 * 0.25 = 0.025$
- 8 [F1 --> F2]:  $P(\text{F} \rightarrow \text{F}) * P(\text{emit T} | \text{F}) = 0.9 * 0.5 = 0.45$
- 9 [F2 --> B3]:  $P(\text{F} \rightarrow \text{B}) * P(\text{emit H} | \text{B}) = 0.1 * 0.75 = 0.075$
- 10 [F2 --> F3]:  $P(\text{F} \rightarrow \text{F}) * P(\text{emit H} | \text{F}) = 0.9 * 0.5 = 0.45$

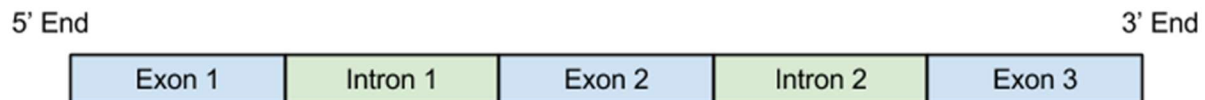
**Calculate (by hand) the dynamic programming table in the Viterbi algorithm for the emitted sequence HTH. Fill in the following table:**

|        | 0     | 1       | 2       |
|--------|-------|---------|---------|
| Biased | 0.375 | 0.08437 | 0.05695 |
| Fair   | 0.25  | 0.1125  | 0.05062 |

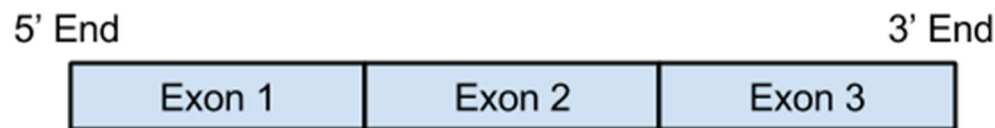
## Finding Donor and Acceptor Splicing Sites

Various bioinformatics tools (such as [GeneMark](#)) use HMMs for gene prediction. Below, we will design a simple HMM aimed at finding exons in a nucleotide sequence.

In a **split gene**, before *messenger RNA* is translated into protein, it undergoes *splicing* to remove some parts of the RNA. The removed intervals are called **introns**, and the remaining intervals (which code for a protein in a protein-coding gene) are called **exons**. Before splicing, a strand of RNA may look like this:



And after splicing it will look like this:



The **donor site** is the region that separates the end of an exon from the beginning of an intron. The **acceptor site** is the region that separates the end of an intron from the beginning of an exon.

The most conservative part of the donor site is the first two nucleotides of introns (usually GU). The most conservative part of the acceptor site is the last two nucleotides of introns (usually AG). We will refer to the first two nucleotides in introns as Donor1 and Donor2 and the last two nucleotides in introns as Acceptor1 and Acceptor2. For example, for canonical donor and acceptor sites GU and AG, Donor1 = G, Donor2 = U, Acceptor1 = A and Acceptor2 = G.

Our goal is to determine whether the genomic region contains exons, and if so, to find them based on a simple (and not very practical) analysis of donor and acceptor sites. Suppose we have done prior analysis of known exons and introns and come up with the following HMM based on the statistical analysis of annotated genes:

1.  $\Sigma: \{A, C, G, T\}$
2. *States*: Exon, Donor1, Donor2, Intron, Acceptor1, Acceptor2
3. *Transitionmatrix*:

|                  | Transition<br>To: <b>Exon</b> | Transition<br>To: <b>Donor1</b> | Transition<br>To: <b>Donor2</b> | Transition<br>To: <b>Intron</b> | Transition To:<br><b>Acceptor1</b> | Transition To:<br><b>Acceptor2</b> |
|------------------|-------------------------------|---------------------------------|---------------------------------|---------------------------------|------------------------------------|------------------------------------|
| <b>Exon</b>      | 0.9                           | 0.1                             | 0.0                             | 0.0                             | 0.0                                | 0.0                                |
| <b>Donor1</b>    | 0.0                           | 0.0                             | 1.0                             | 0.0                             | 0.0                                | 0.0                                |
| <b>Donor2</b>    | 0.0                           | 0.0                             | 0.0                             | 1.0                             | 0.0                                | 0.0                                |
| <b>Intron</b>    | 0.0                           | 0.0                             | 0.0                             | 0.9                             | 0.1                                | 0.0                                |
| <b>Acceptor1</b> | 0.0                           | 0.0                             | 0.0                             | 0.0                             | 0.0                                | 1.0                                |
| <b>Acceptor2</b> | 1.0                           | 0.0                             | 0.0                             | 0.0                             | 0.0                                | 0.0                                |

4. *Emission* matrix:

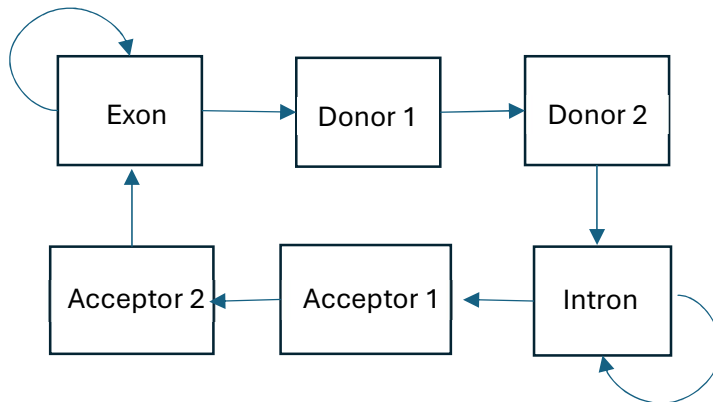
|                  | Emit: <b>A</b> | Emit: <b>C</b> | Emit: <b>G</b> | Emit: <b>T</b> |
|------------------|----------------|----------------|----------------|----------------|
| <b>Exon</b>      | 0.25           | 0.25           | 0.25           | 0.25           |
| <b>Donor1</b>    | 0.05           | 0.00           | 0.95           | 0.00           |
| <b>Donor2</b>    | 0.00           | 0.05           | 0.00           | 0.95           |
| <b>Intron</b>    | 0.40           | 0.10           | 0.10           | 0.40           |
| <b>Acceptor1</b> | 0.95           | 0.00           | 0.05           | 0.00           |
| <b>Acceptor2</b> | 0.05           | 0.00           | 0.95           | 0.00           |

This HMM starts in either the exon or intron state with the same probability. This means that the HMM's starting probabilities are as follows:

| <b>Exon</b> | <b>Donor1</b> | <b>Donor2</b> | <b>Intron</b> | <b>Acceptor1</b> | <b>Acceptor2</b> |
|-------------|---------------|---------------|---------------|------------------|------------------|
| 0.5         | 0.0           | 0.0           | 0.5           | 0.0              | 0.0              |



**Draw the diagram of this HMM (only existing nodes and edges, without numbers).**



We now have a different set of states, observations, transition and emission probability matrices, but the Viterbi algorithm is the same. As a result, we only need to change the input variables located at the top of Viterbi.py. You can copy the lines from [this file](#) to replace the preamble with (or, if you are interested in programming in Python, try to figure out how to change these lines yourself).

The DNA string that we will consider as the emitted symbols of the HMM is found [here](#).

**How many exons and introns do you find in this sequence?**

4 exons and 3 introns

Let's imagine that our colleague conducted an experiment and showed that our test sequence should have many more introns than we found.

**Given this piece of information, how would you change the transition matrix to create an HMM that more accurately finds exons and introns? Be specific about which transitions you would change and why.**

To get more introns, we either need to transition into introns more often or transition out of introns sooner. To do this, we could increase Exon --> Donor 1 (leave an exon and enter a donor site of an intron) or increase Intron --> Acceptor 1 (leave an intron acceptor site and enter an exon). The second option seems counterintuitive until you remember that we are looking for MORE introns, not necessarily longer introns. The sooner we get out of an intron, the quicker we can get back in one.

## Finding CpG Islands

**DNA methylation** typically adds a methyl (CH<sub>3</sub>) group to the cytosine nucleotide within a CG dinucleotide, and methylated C spontaneously deaminate into T over time. As a result, CG is the least frequent dinucleotide in many genomes.

However, methylation is often suppressed near **promoter regions**, where gene transcription is initiated. As a result, these regions form **CG-islands**, where CG appears relatively frequently.

The following **dinucleotide frequency** matrices are obtained based on the statistics of annotated genomic sequences. (It is worth keeping in mind that obtaining these probabilities presents a catch-22; we need the transition probabilities to find the CG-islands, but we must first find the CG-islands in order to determine these frequencies.) Your task is to design an HMM based on these matrices that will help you find the CG-islands.

|   | A     | C     | G            | T     |
|---|-------|-------|--------------|-------|
| A | 0.053 | 0.079 | 0.127        | 0.036 |
| C | 0.037 | 0.058 | <b>0.058</b> | 0.041 |
| G | 0.035 | 0.075 | 0.081        | 0.026 |
| T | 0.024 | 0.105 | 0.115        | 0.050 |
| - | A     | C     | G            | T     |
| A | 0.087 | 0.058 | 0.084        | 0.061 |
| C | 0.067 | 0.063 | <b>0.017</b> | 0.063 |
| G | 0.053 | 0.053 | 0.063        | 0.042 |
| T | 0.051 | 0.070 | 0.084        | 0.084 |

**Figure:** Dinucleotide frequencies for a collection of CG-islands (top) and non-CG-islands (bottom) in the human genome computed for a single strand of the X chromosome. Frequencies of CG are shown in bold.

Your colleague suggests building a simple HMM that consists of just 2 states “+” (CG-island) and “-” (non-CG-island), one emitting symbols in CG-islands, and the other

emitting symbols in non-CG-islands. The figure below shows a sequence of emitted symbols (top) and a possible sequence of hidden states (bottom) for this HMM:

C A C C T T T A T C G C C C T A A T C G C T A G C C G G C

- - - - - + + + + + - - + + + + + + + + + +

**Can you utilize the matrices above to derive emission and transition probabilities for the proposed two-state HMM? Why?**

We cannot directly derive either of the matrices. For the emission matrix: these matrices give the frequencies of dinucleotides, but we don't have the frequencies of the individual nucleotides. For the transition matrix: we do not have the probability of transitioning between island and non-island, we only have probability of being in or out of an island.

After some thought, we decide to use an 8-state HMM rather than a 2-state HMM. The hidden path for this HMM is shown below.

C- A- C- C- T- T- T- A- T+ C+ G+ C+ C+ C+ T+ A- A- T+ C+ G+ C+ T+ A+ G+ C+ C+ G+ G+ C+

The complete description of the 8-state HMM for finding CG-islands is given as follows:

1.  $\Sigma = \{A, C, G, T\}$
2.  $States = \{A+, C+, G+, T+, A-, C-, G-, T-\}$  We partition the set of states into “+” and “-” groups and assume that the starting probabilities across all states are equal to each other.
3. An 8x8 *Transition* matrix. There are two types of transition probabilities. There are transition probabilities *within* the same group (between states with the same sign, e.g., from A+ to C+ or A- to C-) and transition probabilities *between* groups (between states different signs, e.g., from A- to C+). The transition probabilities within groups were calculated from the tables above.

**+ A      C      G      T**

**A** 0.180 0.268 0.430 0.122

**C** 0.191 0.299 0.299 0.211

**G** 0.161 0.346 0.373 0.120

**T** 0.082 0.357 0.391 0.170

**- A C G T**

**A** 0.300 0.200 0.290 0.210

**C** 0.319 0.300 0.081 0.300

**G** 0.251 0.251 0.299 0.199

**T** 0.176 0.242 0.291 0.291

**Explain how we transformed the dinucleotide frequency matrices given previously into the transition probability matrices shown above. (Hint: what is the sum of all transition probabilities for a single state in a single group?)**

The transition matrix is generated by normalizing the values of the dinucleotide matrix such that the row sums are each 1. Normalizing involves taking the row sums in the original matrix and dividing each of the values in the row by their row sum.

As for transition probabilities, we will assume that the probability of transitioning between + and - groups is the same for each nucleotide, as is described in the following tables:

**+ to - A- C- G- T-**

**A+** 0.25 0.25 0.25 0.25

**C+** 0.25 0.25 0.25 0.25

**G+** 0.25 0.25 0.25 0.25

**T+** 0.25 0.25 0.25 0.25

**- to + A+ C+ G+ T+**

**A-** 0.25 0.25 0.25 0.25

**C-** 0.25 0.25 0.25 0.25

**G-** 0.25 0.25 0.25 0.25

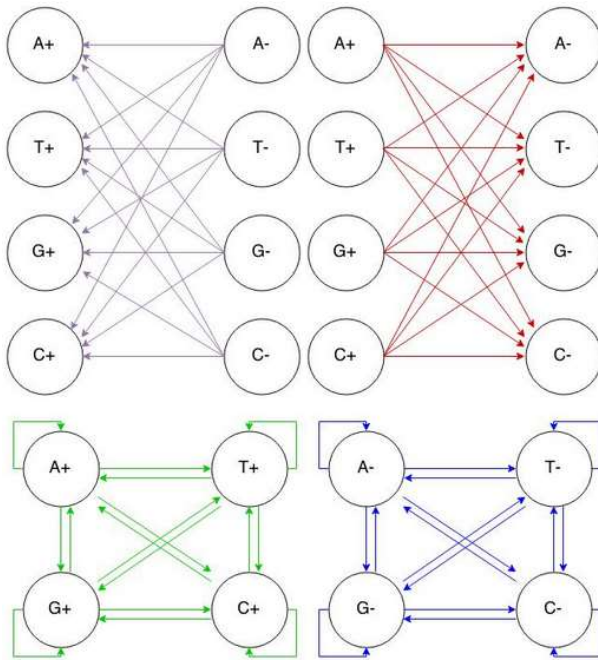
**T-** 0.25 0.25 0.25 0.25

The 4x4 matrices that we already considered regarding transitions within CG-islands (and within non-CG-islands) are used to construct an 8x8 *Transition* matrix (below) using a parameter  $p$ . This parameter is used to ensure that the sum of each row is 1 (without it, the sum of each row would be equal to 2). By varying the value of  $p$ , we weight the preference of group transitions within a group (+ to + **or** - to -) versus transitions that change group (+ to - **or** - to +).

|           | <b>A+</b>       | <b>C+</b>       | <b>G+</b>       | <b>T+</b>       | <b>A-</b>       | <b>C-</b>       | <b>G-</b>       | <b>T-</b>       |
|-----------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|
| <b>A+</b> | $0.180 * (1-p)$ | $0.268 * (1-p)$ | $0.430 * (1-p)$ | $0.122 * (1-p)$ | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      |
| <b>C+</b> | $0.191 * (1-p)$ | $0.299 * (1-p)$ | $0.299 * (1-p)$ | $0.211 * (1-p)$ | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      |
| <b>G+</b> | $0.161 * (1-p)$ | $0.346 * (1-p)$ | $0.373 * (1-p)$ | $0.120 * (1-p)$ | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      |
| <b>T+</b> | $0.082 * (1-p)$ | $0.357 * (1-p)$ | $0.391 * (1-p)$ | $0.170 * (1-p)$ | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      |
| <b>A-</b> | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.300 * (1-p)$ | $0.200 * (1-p)$ | $0.290 * (1-p)$ | $0.210 * (1-p)$ |
| <b>C-</b> | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.319 * (1-p)$ | $0.300 * (1-p)$ | $0.081 * (1-p)$ | $0.300 * (1-p)$ |
| <b>G-</b> | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.251 * (1-p)$ | $0.251 * (1-p)$ | $0.299 * (1-p)$ | $0.199 * (1-p)$ |
| <b>T-</b> | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.25 * p$      | $0.176 * (1-p)$ | $0.242 * (1-p)$ | $0.291 * (1-p)$ | $0.291 * (1-p)$ |

**4.Emission.** This is an unusual HMM in which all emission probabilities are either 1 or 0. Each state emits its “own” nucleotide with probability 1, i.e., G+ always emits G, T- always emits T, etc. The HMM diagram for this HMM is the complete directed graph on 8 nodes, i.e., each node is connected to each other node by an edge (yielding 64 edges overall). The figure below illustrates 16 of these edges corresponding to transitions from “+” to “-” states (left) and 16 edges corresponding to transitions from “-” to “+” states (right). The transition

probability for each of these  $16+16=32$  edges is  $0.25 * p$ , and our goal is to choose the parameter  $p$  that is the most biologically sensible.



To simulate this HMM, we will once again use `Viterbi.py`. As before, we can use `Viterbi.py` by replacing the preamble with the content of [this file](#).

We will assume that the HMM emits a nucleotide string [chr\\_22.txt](#) (which contains the first 1,000 base pairs of human chromosome 22). You need to download this file and save it in the same directory as your (modified) `Viterby.py`. This will ensure that when you run `Viterbi.py`, it is able to read the contents of `chr_22.txt`.

**Run your Python code on the emitted string to get the sequence of hidden states. Do you notice anything troubling about the output?**

There are many zeros in the output matrix, and the probability of the most likely hidden state sequence is 0.

The problem you are likely to encounter when running the previous Python segment is called **underflow**, which occurs when a variable becomes too small for the computer to store in memory (which caused the computer to round it down to zero).

To address the issue of underflow, we will use the Python code [Islands.py](#). Note that all numeric variables were converted (the technical term is “type casted”) to decimal, which tells the computer to allocate more memory for a number. Using the decimal type

effectively tells Python to store more places after the decimal, since the probabilities (scores of nodes) in the Viterbi graph for long sequences can become very small.

In [Islands.py](#), you may change the variable  $p$ , which controls how much weight is given to the transition probabilities within a group versus the transition probabilities between groups. Experiment with different values of  $p$ .

**How many islands do you find with  $p = 0.2$ ? What about  $p = 0.4$ ? Can you guess the number of islands for  $p = 0.9$  without running a program and explain why?**

For  $p=0.2$ , there are 4 islands. For  $p=0.4$ , there are 36 islands. For  $p=0.9$ , there are likely many more islands. The transition probability being high means that + and - will switch off much more often, resulting in many separate instances of + (islands). If the HMM transitions every turn (which would happen for  $p=1.0$ ), we would see 500 islands. So, for 0.9 we would expect somewhere slightly below 500 islands.

## Classifying Worm Proteins

We mentioned that there are over 7,500 known orthologs between *C. elegans* and humans, but biologists are constantly searching for more elusive similarities between other *C. elegans* and human genes. Several large gene families in *C. elegans*, which were first thought to be nematode-specific, have since been classified as putative **G-protein coupled receptors (GPCRs)**. These proteins, which are likely to have orthologs in other species, are receptors weaved into the cell membrane that sense extracellular stimuli and activate inside signal transduction pathways that trigger cellular responses. The human genome encodes at least 350 GPCRs, and the function of most of these proteins remains unknown.

Because protein function largely depends on protein structure (which is often difficult to infer from amino acid sequence alone), detecting function similarity between these nematode sequences and known GPCRs in other organisms is a nontrivial task. However, if we can find human proteins that are similar in function to these nematode sequences, then we will be able to study the nematode counterpart in a laboratory and extrapolate the experimental results to humans (with some further experiments, of course).

Here we choose the putative GPCR gene **sra-4** in *C. elegans* (**Serpentine Receptor A**, Wormpep AH6.8; UniProt Q09206; 329 aa) as an example. Your goal is to find a protein of known function in humans, which may serve as a possible ortholog of sra-4.

Before we can analyze sra-4, we must first retrieve its sequence. UniProt, the “Universal Protein Resource”, is a comprehensive resource for protein sequence and annotation data. From UniProt, we can obtain the sequence of a protein as well as its annotation.

Go to [UniProt](#) and search for the protein with ID Q09206. Read the "Function" section - it contains terms from the Gene Ontology (GO) database, applied to the protein. Check if this protein is a GPCR, and determine the processes in which it is involved.

Then, download the protein in FASTA format.

**What is the length of this protein (in amino acids)?**

329 amino acids



### Initial BLAST search of sra-4 protein.

“BLAST,” the “Basic Local Alignment Search Tool,” is a tool for finding similarities between a query sequence and sequences in a protein database. It is effectively the “Google search” of biological sequences: much like how we can “Google” a key term of interest, we can “BLAST” a biological sequence of interest to find similar sequences. BLAST performs a local alignment of the query sequences against sequences in the target database and returns the results in the order of ascending **E-values**. The “E-value” (or “Expected value”) is the number of hits with the same or higher score one would “expect” to see by chance when searching a randomly generated (decoy) database of the same size. Therefore, a smaller E-value corresponds to a more statistically significant match.

To quickly find any proteins similar to sra-4, you can run a BLASTP search (where the “P” stands for “Protein”) against NCBI BLAST’s “non-redundant (nr) protein sequences” database using the [NCBI BLAST server](#).

**Considering the first 10 matches, how many hits do you find with an E-value equal to zero? How many among the first ten matches are *C.elegans* genes? Did you find any non-worm proteins (other than the *Caenorhabditis* or *Ancylostoma* genus) among the first 50 hits?**

Of the first 10 matches, the top 2 have E-value = 0 and the top 5 are from *C. elegans*. All of the top 50 hits are from the genus *Caenorhabditis*.

### Trying to find human orthologs directly

In the previous exercise, we identified a large number of proteins similar to sra-4, and almost all of the top hits belonged to the same species or to close relatives. This is a fairly natural result, but our goal is to find human proteins. Recall that BLAST’s “non-redundant (nr) protein database” contains proteins from various species, including humans, and we can limit our search within certain taxonomic groups. To restrict search, run [BLAST](#)

for sra-4 again, filling in the optional “Organism” field with “Homo sapiens”.

**15. How many significantly similar sequences (e-value < 0.01) do you find?**

None

## Constructing a profile HMM from the multiple alignment and using it to search a database

We can see that we need a different approach to search for human orthologs of worm genes. As we already know, we can build a HMM to find more distant similarities by looking for specific signatures in proteins.

We will try to use **HMMER**, which is a toolkit that performs protein analysis using profile HMMs. We will use HMMER to construct a profile HMM of the multiple alignment and to search it against the protein database. HMMER contains the following search tools from which we must choose:

- phmmer: Searching a protein sequence against a protein database by building a profile HMM from your single query sequence and searching this profile HMM against a protein sequence database
- hmmsearch: Searching a protein alignment or profile HMM against a protein sequence database. If a protein alignment is input, a profile HMM will be generated from the alignment that will be used to search against the protein sequence database.
- jackhammer: Iterative version of phmmer, but iterative: similar sequences detected in a previous round are incorporated into a new profile, and the new profile is searched again until there are no improvements (or little improvements) in score.

Because we have only one sra-4 sequence, we will use [phmmer](#)

. Paste the downloaded sequence of sra-4 in FASTA format (or upload the file) in the main window. In the field "Sequence Database" select UniProtKB (Universal Protein Resource KnowledgeBase). Select the field "Restrict by taxonomy" and enter *Homo sapiens* here.

### Do you find any significant hits?

No significant hits.

Should we give up? Of course not! Just because the methods that we have used have not identified a similarity does not mean that one does not exist.

There is a large collection of protein families and corresponding multiple sequence alignments, called **Pfam** (**P**rotein **F**amilies), which we can check for similarity with sra-4.

Use [hmmsearch](#) to search against the Pfam database of profile HMMs using the sra-4 sequence. Paste the sequence or upload the file in the main window, and leave all other parameters as their default values. Check for the significance of identified matches (be sure that e-value < 0.01). Select the most significant result (click on its Id), and see the protein description page.

### **What is the name of the corresponding family?**

7TM (transmembrane) Serpentine type G-protein coupled receptors Sra  
(superfamily of chemoreceptors)

Now that we have identified the most related family, we need to obtain some representatives of this family to align and use as a model for the subsequent steps. Fortunately, an alignment for each Pfam family has already been constructed.

Click on the Family ID of the most related family. This will take you to a Pfam page for the family. Select "Alignments" (from the menu on the left side of the page), go to "Download options" on the bottom of page, and download the set of NCBI proteins in raw Stockholm format.

### **How many proteins are in this dataset?**

877 proteins

Now that we have an alignment of sequences from a protein family, it is time to use [hmmsearch](#). This tool takes a protein alignment as an input, and it generates a profile HMM from the alignment that it uses to search against a specified protein sequence database.

As with many bioinformatics tools, hmmsearch is very sensitive to the input file format. In the Pfam database, the alignment is stored along with additional information, so in order to avoid errors, open the downloaded file in any text editor and remove all strings before the second occurrence of "# STOCKHOLM 1.0". The beginning of the new file should look like the following:

```
# STOCKHOLM 1.0  
  
OZG03163.1/40-299
```

Open hmmsearch, paste or upload your edited alignment in the main window, and restrict your search by taxonomy with *Homo sapiens* and select UniProtKB as the database (as before).

We will relax our notion of significance to consider a gene with e-value less than 0.05.

**Give the name of the most significant gene identified.**

Prostaglandin E2 receptor (EP2 subtype)

Since we found the human gene that fits the HMM profile based on the sra-4 sequence, we can investigate its structure and function. Click on the protein name, which will open the corresponding page in the UniProt database.

**Try to find additional information about this gene and try to answer the following:**

- **where in the cell is it located, and how does its location relate to its structure and function?**
- **in which processes in the organism is it involved?**

The protein is transmembrane, which makes sense with our hypothesis. It is therefore involved in extracellular signal reception and internal signal transduction. Other than its function as a GPCR, it has been implicated in inflammation, cell proliferation, and response to lipopolysaccharide and progesterone.

## **Conclusion**

The power of HMMs has allowed us to compare very distant organisms, which were separated so long ago that direct alignment fails to find any matches between them. From the set of aligned sequences we obtained a profile HMM, which contains the signatures of the protein of interest. We then got the opportunity to look for proteins with similar signatures in any organism, even very distant ones.

We have determined that sra-4, which is a member of the seven-transmembrane G-protein-coupled receptor class (7TM GPCRs), has a close relative in the human proteome. Sra-4 belongs to a family of chemoreceptors, which are a very important class of molecules for the worms. Chemoperception is one of the central "senses" of nematodes like *C. elegans*, which are otherwise blind and deaf. We have more advanced senses, but we still use almost the same structure to transduce chemical signals inside our bodies.